

Design Document: Protein Folding (HP Lattice Model) Domain for DeepXube

J.C. Vaught

April 2026

1 Overview

This document specifies a new domain for the DeepXube framework that encodes protein folding on a three-dimensional cubic lattice under the hydrophobic-polar (HP) model. A chain of N residues, each labeled H (hydrophobic) or P (polar), is embedded on $\mathbb{Z}^3$ as a self-avoiding walk. The task is to reconfigure one valid conformation into another that matches a target contact pattern, using local lattice moves as discrete actions. The learned heuristic $h(\text{state}, \text{goal}) \rightarrow \hat{c}$ estimates cost-to-go for A* search.

The domain is registered with `@domain_factory.register_class("hplattice")` and inherits from `ActsEnum`, `GoalStartRevWalkable`, `HasFlatSGIn`, `StateGoalVizable`, and `StringToAct`, following the established pattern of the robotic arm and retrosynthesis domains.

2 State Representation

The state encodes the full three-dimensional conformation of the chain. Two arrays define it completely.

`HPState` stores a positions array `coords` of shape $(N, 3)$ with dtype `int16`, giving the absolute lattice coordinates (x_i, y_i, z_i) of each residue $i \in \{0, \dots, N-1\}$. The sequence identity (which residues are H and which are P) is stored once in the domain object, not in the state, since it never changes during folding. The state also caches its hash for use in A*'s CLOSED set.

Self-avoidance is enforced as an *invariant*: no two residues may occupy the same lattice site, and consecutive residues must be lattice-adjacent (Manhattan distance exactly 1). Every transition function verifies this before returning a successor state. Invalid moves produce no state change, which is handled by action filtering in `get_state_actions`.

A secondary derived representation used for the neural network input encodes the conformation as a flat `int8` array of length $2(N-1)$. The first $N-1$ entries store relative direction indices (0–5 mapping to $+x, -x, +y, -y, +z, -z$) between consecutive residues. The remaining $N-1$ entries store the sequence type of each residue from index 1 onward (0 for P, 1 for H), providing the network with both structural and chemical context. Combined with the one-hot encoding in the `HasFlatSGIn` mixin, the network input becomes shape $(B, 2(N-1))$ with one-hot depths of 6 for directions and 2 for residue types.

`__hash__` and `__eq__` operate on `coords.tobytes()`, following the Cube3 and NPuzzle conventions.

3 Goal Representation

The goal is a target *contact map*: an upper-triangular binary matrix $C^* \in \{0,1\}^{N \times N}$ recording which pairs of H residues are lattice-adjacent (Manhattan distance 1) but not sequentially adjacent ($|i - j| > 1$). For a chain of length N , the contact map is stored as a flat `uint8` array of length $N(N-1)/2$ (the upper triangle), yielding shape $(B, N(N-1)/2)$ for batched neural network input. Only H-H contacts are semantically meaningful; entries involving P residues are always zero.

This representation is strictly more informative than a target energy, because multiple distinct contact maps can yield the same energy. It is also more general than requiring an exact target conformation, since multiple conformations (related by translation and rotation) can produce the same contact map. This controlled ambiguity is desirable: the goal set contains all conformations with the specified H-H adjacency pattern.

For `get_input_info_flat_sg`, the state dimension is $2(N-1)$ with one-hot depths $[6, 2]$, and the goal dimension is $N(N-1)/2$ with one-hot depth 1 (raw binary, no one-hot needed).

4 Actions

Four types of local lattice moves are defined, yielding a variable number of valid actions per state (hence `ActsEnum`, not `ActsEnumFixed`).

End moves (2 per endpoint). The first or last residue is moved to any unoccupied lattice neighbor of its chain-adjacent partner. Each endpoint has up to 4 valid target sites (5 neighbors minus the one occupied by residue 1 or $N-2$, minus any occupied by other residues). This gives up to 8 candidate actions across both ends.

Corner moves. For an internal residue i where $\mathbf{r}_{i-1}$ and $\mathbf{r}_{i+1}$ are not collinear (i.e., the chain makes a bend), residue i can be moved to the unique lattice site that maintains adjacency with both $i-1$ and $i+1$ while changing the corner direction. The new site must be unoccupied. This produces up to 1 valid move per eligible internal residue, for a maximum of $N-2$ corner move candidates.

Crankshaft moves. When two consecutive internal residues $i, i+1$ form a U-shape with their neighbors (i.e., $\mathbf{r}_{i-1}, \mathbf{r}_i, \mathbf{r}_{i+1}, \mathbf{r}_{i+2}$ lie in a plane and form a crankshaft pattern), both i and $i+1$ are rotated 180 degrees about the axis connecting $\mathbf{r}_{i-1}$ and $\mathbf{r}_{i+2}$. Both target sites must be unoccupied. Up to $N-3$ candidates.

Pull moves. Residue i is moved to an unoccupied site adjacent to $i+1$'s current position; then $i-1$ is pulled to i 's former position if it is adjacent, otherwise the chain propagates backward. Pull moves are the most general and ensure ergodicity (any conformation is reachable from any other on a sufficiently large lattice). Each internal residue can have up to 4 pull move candidates.

The total number of candidate actions per state is $O(N)$. After filtering for self-avoidance and chain connectivity, the typical number of *valid* actions for a 20-residue chain is roughly 15–40. Each action is represented as an `HPAction` object encoding the move type (2 bits), the residue index involved (up to 7 bits for $N \leq 127$), and the target direction.

The transition cost for every valid action is 1.0, making cost-to-go equivalent to the number of moves.

5 Transition Dynamics and Non-Decomposability

Applying an action updates the `coords` array by rewriting one or two rows (the moved residues) and returns the new state with transition cost 1.0. The `next_state` method copies the coordinate

array, applies the positional update, and validates self-avoidance.

This domain is non-decomposable for two fundamental reasons. First, self-avoidance creates global coupling: whether residue i can move to a target site depends on the positions of all other residues, not just its neighbors. A distant residue can block a critical move. Second, H-H contacts are non-local in sequence but local in space. Changing the fold at one end of the chain can create or destroy contacts involving residues far away in sequence, making it impossible to optimize sub-chains independently.

6 is_solved Check

A state is solved when its contact map matches the goal contact map exactly. The check computes the current contact map from `coords`: for every pair (i, j) with $|i - j| > 1$ where both residues are H, test whether $\|\mathbf{r}_i - \mathbf{r}_j\|_1 = 1$. The resulting binary vector is compared element-wise against the goal. This runs in $O(N^2)$ time, which is acceptable for the chain lengths considered (up to $N = 36$).

An energy-threshold alternative was considered and rejected, because multiple contact maps can share the same energy, creating ambiguity about which fold is “correct.” Exact contact-map matching provides a clean, unambiguous solved criterion.

7 Energy Computation

The HP energy of a conformation is $E = -\sum_{(i,j):|i-j|>1} \mathbf{1}[\text{type}(i) = \text{H}] \cdot \mathbf{1}[\text{type}(j) = \text{H}] \cdot \mathbf{1}[\|\mathbf{r}_i - \mathbf{r}_j\|_1 = 1]$. This counts non-sequential H-H lattice contacts and negates the sum. For $N = 20$, the pairwise check involves $\binom{20}{2} = 190$ pairs; for $N = 36$, it is 630 pairs. Using vectorized NumPy operations on the $(N, 3)$ coordinate array, energy computation takes approximately $5 \mu\text{s}$ for $N = 20$, well within the per-step budget.

Energy is not used as the transition cost (which is always 1.0) but can be used as an auxiliary training signal or for visualization. The contact map used in the goal encodes the full energy breakdown: $E = -\sum C_{ij}^*$.

8 Visualization

The visualization uses matplotlib’s 3D projection with the brand color palette and sharp, geometric aesthetics. The layout is a side-by-side comparison: the current conformation on the left, the target (goal) conformation on the right, rendered on a shared camera angle.

The chain backbone is drawn as a thick, straight-edged tube connecting lattice sites, with no rounded caps. Each residue is rendered as a cube placed on its lattice site. Hydrophobic (H) residues are colored in **Garnet #73000A**, and polar (P) residues in **Atlantic #466A9F**. Chain connectivity is shown with Congaree (**#1F414D**) cylindrical segments.

H-H contacts that exist in the current fold are highlighted with thick Horseshoe (**#65780B**) bonds connecting the relevant cubes. Contacts present in the target but missing from the current fold are rendered as translucent Rose (**#CC2E40**) dashed lines (ghost bonds), making it visually clear which contacts still need to be formed. Contacts present in both current and target are drawn in bright Grass (**#CED318**).

The background is set to 10% Black (**#ECECEC**). Lattice gridlines are drawn at 10% opacity in 50% Black (**#A2A2A2**). A text bar at the bottom shows energy, contact count, and solved status. For GIF export, the camera slowly rotates at 5 degrees per frame over 72 frames, producing a

smooth 360-degree orbit. Each frame also updates the fold as the A* solution sequence is replayed, showing contacts “clicking into place” one move at a time.

9 Training Considerations

sample_goalstate_goal_pairs. Generate a random valid self-avoiding walk of length N on $\mathbb{Z}^3$ using pivot algorithm sampling (which produces uniformly distributed conformations). Compute the contact map of the generated conformation and return the (state, goal) pair. The pivot algorithm generates a candidate by choosing a random residue and rotating the sub-chain beyond it by a random lattice symmetry (one of 48 rotations in the cubic group); the candidate is accepted if the resulting walk is self-avoiding. After a burn-in of $O(N)$ pivots, the distribution is approximately uniform over all N -step self-avoiding walks.

random_walk_rev. From a goal state, take k random valid moves (sampled via `get_state_actions` and uniform selection) to produce a start state that is approximately k moves away from the goal. Since the move set is not trivially reversible (pull moves are directional), this method uses forward random walks from the goal state as a proxy for reverse walks, following the `GoalStartRevWalkable` pattern. The correlation between k steps and true distance is noisy but sufficient for training, as established by the framework’s existing use of this technique for the robotic arm domain.

Chain lengths. Recommended progression: $N = 10$ (trivial, for unit testing), $N = 16$ (easy benchmark), $N = 20$ (standard HP benchmark from the literature, approx. 10^{11} conformations), $N = 27$ (the “Harvard” sequence, a well-known challenge), $N = 36$ (hard, state space approximately 10^{18}). The domain constructor accepts N and the HP sequence string as parameters. It is registered under the name `hplattice`, e.g., `hplattice.20_HPHPPH...`

State space size. The number of 3D self-avoiding walks of length N grows as $\mu^N N^{\gamma-1}$ where $\mu \approx 4.684$ is the connective constant for the cubic lattice and $\gamma \approx 1.157$. For $N = 20$, this gives roughly $4.684^{20} \approx 4 \times 10^{13}$ conformations. The effective state space explored by local moves is smaller due to connectivity, but remains vast.

10 Challenges and Tradeoffs

Self-avoidance filtering. The primary computational cost is checking whether candidate move targets are occupied. A hash set of occupied positions reduces this to $O(1)$ per query, but the set must be rebuilt or updated for each state. For $N \leq 36$, maintaining a `frozenset` of coordinate tuples is fast enough; for larger chains, a spatial hash grid would be needed.

Dead ends. Certain conformations are topologically trapped: all valid local moves lead to states that are farther from the goal. This is rare for short chains but becomes significant for $N > 25$. Pull moves mitigate this by guaranteeing ergodicity, but they increase the branching factor. The tradeoff is between completeness (including pull moves) and search efficiency (excluding them for tighter action spaces).

Sampling valid conformations. Naive random placement (choosing random directions) produces self-intersecting walks with high probability for $N > 15$. The pivot algorithm solves this but requires $O(N)$ burn-in steps. An alternative is to use biased random walks with backtracking (Rosenbluth sampling), which is faster but produces a non-uniform distribution that must be corrected with importance weights.

Goal ambiguity. Multiple conformations satisfy the same contact map (related by global rotation, reflection, and certain internal rearrangements). This is intentional and beneficial: the

heuristic learns a distance function over equivalence classes of conformations rather than to a single target, improving generalization. However, it means the search may find a solution conformation that is geometrically different from the one used to generate the goal, which is correct behavior but complicates visualization.

Non-reversibility. Unlike the Rubik’s Cube or N -puzzle, local lattice moves are not trivially invertible. A corner move at residue i can be reversed by another corner move at i , but pull moves shift multiple residues and have no simple inverse. This means `random_walk_rev` must approximate reverse walks with forward walks, introducing noise into the training signal. Empirically, this works well for the robotic arm domain and is expected to transfer.

Scalability. For $N = 36$, the branching factor averages around 50 and the solution depth can exceed 100 moves. With a learned heuristic, A* should remain tractable if h is reasonably accurate, but training will require millions of problem instances and deep random walks (up to $k = 200$ scramble steps). GPU-batched heuristic evaluation is essential for search performance.